When Does Monitorability-Aware Process RL Help? A Tiny Symbolic Study of Budgeted Verification

Anonymous Author(s)
Affiliation
Address
email

Abstract

We study when monitorability-aware process rewards help a tiny controller allocate verification under a strict reasoning budget. Our environment is a procedurally generated arithmetic chain with exact local checking, one optional hidden faulty step, backtracking, and a noisy one-bit ambiguity cue. A controller must choose among `THINK`, `VERIFY`, `REVISE`, and `COMMIT` while observing only compact partial state. We compare random play, heuristics, outcome-only RL, naive process rewards, and a monitorability-aware reward using a 5,124-parameter masked Double DQN trained locally on an Apple M1 laptop. On train-like depths, the proposed reward is the strongest learned method: it reaches 0.601 ± 0.052 success versus 0.234 ± 0.099 for outcome-only RL, and it learns cue-conditioned verification ($P(\texttt{VERIFY} \mid c{=}1) = 0.381 \pm 0.030$ versus 0.142 ± 0.034 for $c{=}0$). But the policy remains brittle under longer held-out chains: held-out success drops to 0.112 ± 0.035, below outcome-only RL at 0.132 ± 0.055 and far below a simple hint-triggered heuristic at 0.819 ± 0.019. Cleaner cues improve diagnostics more than success, while noisier cues erode both. The main contribution is therefore mechanistic rather than algorithmic: monitorability-aware shaping can induce ambiguity-focused checking in-distribution, but that behavior does not become a robust longer-horizon repair policy.

1 Introduction

Recent reasoning-model work emphasizes that both train-time and test-time compute can matter for hard reasoning [OpenAI, 2024, DeepSeek-AI et al., 2025]. At the same time, recent monitorability work argues that chain-of-thought traces can expose failures better than outputs alone, but that this signal may trade off against capability and may itself be fragile [OpenAI, 2025]. Anthropic's faithfulness results sharpen the caution: even when a reasoning trace is available, it need not be a faithful account of what the model actually used [Anthropic, 2025]. These observations motivate a smaller question than "does process supervision help frontier models?": under a strict reasoning budget, when does process-aware RL help a tiny controller allocate verification effort, and when does it overfit?

We study that question in a deliberately small symbolic environment. The goal is not to chase benchmark scores or to compare against frontier language models. Instead, we construct a tiny partially observable arithmetic-chain environment with exact local checking, one optional hidden faulty transition, backtracking, a finite budget, and a noisy local ambiguity cue. The controller sees only a compact observation: normalized progress, remaining budget, a budget-slack feature, an unchecked-gap feature, the current ambiguity cue,

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

whether it just revised, and the last verification status. This setup makes every relevant quantity exact: correctness, local fault hits, verification precision/recall, cue-conditioned verification rates, and failure localization can all be computed directly from environment state.

We organize the empirical study around three falsifiable questions. First, does monitorability-aware shaping increase *selective* verification at ambiguous steps, rather than just increasing verification volume? Second, does any gain depend on the quality of the local ambiguity cue? Third, if a selective policy is learned in-distribution, does it survive a length shift to longer chains? These questions are small enough to answer exactly in the symbolic setting, but still map onto the broader reasoning-RL themes of budgeted test-time compute, process supervision, and monitorability.

The core result is a structured failure mode. A simple hand-coded hint-triggered heuristic, using only the observable ambiguity cue, solves both training-depth and held-out longer chains well, so the environment does admit a strong selective-verification policy. Outcome-only RL does not reliably discover it. The proposed monitorability-aware reward does learn a more diagnostic in-distribution policy than outcome-only or naive process rewards, but this policy remains brittle under a depth shift from 3–6 training steps to 7–9 held-out steps. The resulting paper is therefore best read as a concept-and-feasibility plus negative-result study: a tiny exact proxy can isolate ambiguity-focused checking behavior, but small-budget RL still fails to recover a robust version of that behavior.

Concretely, the paper contributes four pieces: a tiny exact environment for budgeted process reasoning with local ambiguity cues, verification, and repair; a simple monitorability-aware reward that explicitly values informative verification and bounded unchecked gaps; behavior-level diagnostics that measure whether a policy actually verifies more often on flagged ambiguous steps; and an empirical study showing that in-distribution ambiguity checking can improve while out-of-distribution robustness remains poor.

2 Related Work

OpenAI [2024] and DeepSeek-AI et al. [2025] motivate reinforcement learning for reasoning and highlight the role of train-time and test-time compute. Xie et al. [2025] argue that small, verifiable symbolic environments can reveal reasoning dynamics cheaply. We follow that spirit, but focus on a much smaller controller and a fixed budgeted control problem rather than a large language model.

For monitorability, OpenAI [2025] distinguish intervention, process, and outcome-property evaluations and emphasize a capability-monitorability tradeoff. Our environment is a small process-style proxy in which the trace determines whether failures remain locally diagnosable. Finally, Anthropic [2025] show that reasoning traces need not faithfully reveal the true internal cause of a model's output. Our study does not solve faithfulness; instead, it asks whether even a perfectly instrumented symbolic proxy already exhibits a tension between solving the task and keeping failure traces locally monitorable.

3 Problem Setup

Each episode samples an arithmetic chain of depth d. Training uses $d \in \{3, \dots, 6\}$; held-out evaluation uses $d \in \{7, \dots, 9\}$. The chain is generated procedurally, so no dataset is stored. With probability p_{fault}, exactly one transition $h \in \{1, \dots, d\}$ is faulty. Conditional on a fault, its location is drawn from a middle-peaked prior. In addition, each step carries an observable binary ambiguity cue $c_t \in \{0, 1\}$ for the current local transition. This cue is noisy: the true faulty step is flagged with high probability, but non-fault steps can also be flagged spuriously.

The controller observes only a compact partial state

$$o_t = \big(\hat{s}_t,\ \hat{b}_t,\ \hat{d},\ \hat{u}_t,\ \hat{g}_t,\ c_t,\ \rho_t,\ \ell_t\big), \tag{1}$$

where $\hat{s}_t$ is normalized progress, $\hat{b}_t$ remaining budget, $\hat{d}$ normalized target depth, $\hat{u}_t$ normalized budget slack, $\hat{g}_t$ the normalized gap since the last checked step, ρ_t indicates whether

the previous action was `REVISE`, and $\ell_t \in \{\text{none}, \text{safe}, \text{unsafe}\}$ is the most recent verification outcome. The latent arithmetic state and the true fault location are never observed directly.

Available actions are `THINK`, `VERIFY`, `REVISE`, and `COMMIT`. `THINK` advances one step. `VERIFY` checks the current local transition and returns a trustworthy safe/unsafe signal. `REVISE` moves one step backward. `COMMIT` ends the episode. If the controller commits at depth d with the correct latent state, the episode succeeds; otherwise it fails.

The base reward is

$$r_t = \mathbb{1}[a_t = \texttt{COMMIT} \wedge \text{correct}] - 1.2\,\mathbb{1}[a_t = \texttt{COMMIT} \wedge \neg\text{correct}] - \lambda_c\, c(a_t) + r_t^{\text{proc}}, \quad (2)$$

where $c(a_t)$ is the action cost and λ_c is the per-action budget penalty.

We report success rate, average return, average action counts, verification precision and recall with respect to the true hidden fault, cost-adjusted accuracy, and a failed-episode localization score

$$\text{Loc} = \begin{cases} 0, & \tau_{\text{diag}} < h, \\ 1 - \dfrac{\tau_{\text{diag}} - h}{\max(1, d - h)}, & \tau_{\text{diag}} \geq h, \end{cases} \quad (3)$$

where h is the true faulty step and τ_{diag} is the first step at which the trace either explicitly localizes the fault through an unsafe verification or ends at commit. High Loc means that failures become diagnosable near their true source rather than remaining opaque.

4 Method

We compare three learned reward schemes. Outcome-only RL uses $r_t^{\text{proc}} = 0$. Naive process RL adds a small bonus for locally valid progress and first-time useful verification, without distinguishing between high- and low-value checks. This makes it prone to weak or poorly targeted checking.

Our proposed monitorability-aware process term is

$$\begin{aligned} r_t^{\text{proc}} = {} & \lambda_{\text{safe}}\,\mathbb{1}[v_t^{\text{safe}}] + \lambda_{\text{fault}}\,\mathbb{1}[v_t^{\text{fault}}] \\ & + \lambda_{\text{hint}}\,\mathbb{1}[a_t = \texttt{VERIFY} \wedge c_t = 1 \wedge \neg w_t] \\ & - \lambda_{\text{waste}}\,\mathbb{1}[w_t] - \lambda_{\text{gap}}\,\frac{g_t - g_{t-1}}{\max(1, d)}, \end{aligned} \quad (4)$$

Here v_t^{safe} denotes a first-time safe verification on a locally valid step, v_t^{fault} a first-time unsafe verification that hits the true faulty transition, w_t a repeated or otherwise uninformative verification, and g_t the unchecked gap between the current step and the most recently checked step. The ambiguity-aware term rewards spending verification budget on locally flagged ambiguous steps rather than on uniformly low-signal checks.

RL uses a tiny masked Double DQN with a two-layer MLP and discrete action masking. Let $Q_\theta(o, a)$ be the online network and $Q_{\bar{\theta}}$ the target network. We minimize the standard one-step temporal-difference loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\big(Q_\theta(o_t, a_t) - y_t\big)^2\Big], \qquad y_t = r_t + \gamma(1 - d_t)\, Q_{\bar{\theta}}\Big(o_{t+1}, \arg\max_{a'} Q_\theta(o_{t+1}, a')\Big), \quad (5)$$

with $\gamma = 0.99$. The network is intentionally tiny: the default controller has 5,124 trainable parameters. The point of the study is reward design and failure analysis under a small compute budget, not scaling.

5 Experimental Setup

The default environment uses training depths 3–6, held-out depths 7–9, fault probability $p_{\text{fault}} = 0.8$, unit action costs, and a budget equal to depth plus six extra actions. The ambiguity cue flags the true faulty step with probability 0.85 and produces false positives on non-fault steps with probability 0.15. We evaluate random play, `Always THINK`, `Verify Every Step`, a hand-coded `Hint-Triggered Heuristic`, outcome-only RL, naive process

RL, and monitorability-aware RL. We also train four ablations of the proposed method: no budget penalty, no `VERIFY`, no `REVISE`, and no monitorability extras. The last ablation removes the cue-conditioned bonuses/penalties and unchecked-gap penalty while retaining only the basic safe/fault process bonuses.

The full experiment uses five random seeds unless otherwise stated, with 300 procedurally generated evaluation episodes per seed and setting. In total, the paper trains 35 learned models (7 learned variants times 5 seeds), each for 5,000 episodes, and evaluates 198,000 episodes across 12 settings, 11 methods, and 5 seeds. We report mean ± standard error over seeds throughout. All runs are local and CPU-compatible; the reported experiments were executed on a MacBook Air with an Apple M1 chip. The repository remains well under the 1 GB footprint target and requires no external datasets, pretrained models, or cloud resources.

6 Results

6.1 A strong selective policy exists

The first important sanity check is that the environment does support a strong policy. The hint-triggered heuristic, which uses only the observable ambiguity cue and no privileged access to hidden state, reaches 0.827 ± 0.012 success on train-like depths and 0.819 ± 0.019 on held-out longer chains. This matters because it rules out a trivial explanation for RL failure: the task itself is not impossible under the imposed budget. Selective ambiguity-focused verification is available; the learned policies simply do not recover it robustly.

6.2 Monitorability-aware RL helps in-distribution

On train-like depths, the monitorability-aware reward is the strongest learned method. It reaches 0.601 ± 0.052 success, compared with 0.234 ± 0.099 for outcome-only RL and 0.187 ± 0.048 for naive process RL. It also checks much more effectively than outcome-only RL, with verification recall 0.836 ± 0.012 versus 0.318 ± 0.131, and it raises failed-episode localization from 0.259 ± 0.116 to 0.675 ± 0.057. Relative to the naive process reward, it not only succeeds more often but also localizes failures more cleanly (0.675 versus 0.152). This is the main positive result of the paper: explicit ambiguity-aware process rewards can induce short-horizon checking behavior in a tiny controller.

The new behavior-level diagnostics show that this is not just more verification volume. On train-like depths, monitorability-aware RL verifies much more often when the cue indicates local ambiguity than when it does not: $P(\texttt{VERIFY} \mid c{=}1) = 0.381 \pm 0.030$ versus 0.142 ± 0.034 for $c{=}0$ (Appendix Figure 4). Outcome-only RL is not selective by the same measure (0.261 ± 0.066 versus 0.279 ± 0.057), while the hint-triggered heuristic is the idealized extreme (0.464 ± 0.002 versus 0.000). The monitorability reward therefore changes *where* the controller spends verification budget, not only how much of it it spends.

Figure 1 and Table 1 summarize the main comparison. Two patterns matter more than raw score. First, monitorability-aware RL is much more diagnostic than outcome-only RL: it checks more often, localizes failures earlier, and attains much higher verification recall. Second, a large gap remains between the learned policy and the simple hint-triggered heuristic, which means the learned controller is still only partially exploiting the available cue.

The ablations sharpen this point. On train-like depths, the full proposed reward is stronger than its monitorability ablations: removing the budget penalty lowers success to 0.479 ± 0.077, removing the monitorability extras lowers it to 0.387 ± 0.087, removing `REVISE` lowers it to 0.186 ± 0.009, and removing `VERIFY` collapses to zero. In-distribution, the complete reward is therefore not just different but genuinely useful.

6.3 The gain is brittle under length shift

The main negative result appears under the depth shift from 3–6 training steps to 7–9 held-out steps. The monitorability-aware policy keeps much better diagnostic behavior than outcome-only RL, with failure localization 0.556 ± 0.083 versus 0.119 ± 0.054 and verification

Table 1: Main metrics on train-like and held-out longer depths; entries are mean ± standard error over five seeds.

Method	TrainSucc	TrainLoc	HeldSucc	HeldLoc	HeldCostAdj
Random	0.005 ± 0.001	0.097 ± 0.005	0.000 ± 0.000	0.042 ± 0.003	-0.085 ± 0.005
Always THINK	0.203 ± 0.008	0.074 ± 0.004	0.203 ± 0.008	0.030 ± 0.003	-0.199 ± 0.007
Verify Every Step	0.565 ± 0.007	1.000 ± 0.000	0.055 ± 0.005	0.981 ± 0.003	-0.646 ± 0.006
Hint-Triggered Heuristic	0.827 ± 0.012	0.356 ± 0.021	0.819 ± 0.019	0.342 ± 0.028	0.241 ± 0.017
Outcome-only RL	0.234 ± 0.099	0.259 ± 0.116	0.132 ± 0.055	0.119 ± 0.054	-0.194 ± 0.062
Naive Process RL	0.187 ± 0.048	0.152 ± 0.083	0.070 ± 0.026	0.163 ± 0.055	-0.238 ± 0.074
Monitorability-aware RL	0.601 ± 0.052	0.675 ± 0.057	0.112 ± 0.035	0.556 ± 0.083	-0.441 ± 0.048

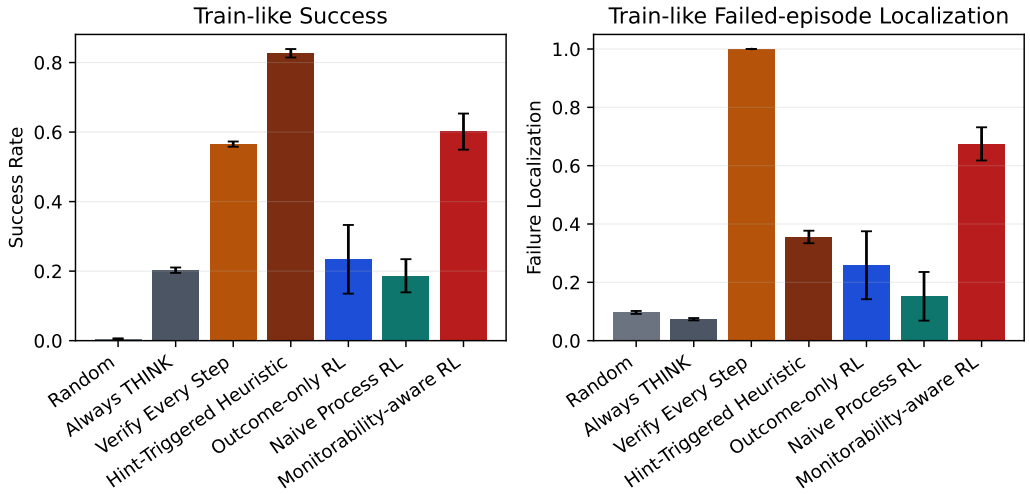


Figure 1: Main train-like comparison. Error bars show standard error over five seeds. A strong hand-coded hint-triggered policy exists; monitorability-aware RL improves checking behavior relative to learned baselines but does not close the gap.

recall 0.555 ± 0.080 versus 0.123 ± 0.060. But it does not preserve its train-like success: held-out success falls to 0.112 ± 0.035, slightly below outcome-only RL at 0.132 ± 0.055 and far below the hint-triggered heuristic at 0.819 ± 0.019. Instead, the learned policy spends budget on ambiguity-focused checking without recovering a robust long-horizon repair policy.

This creates a clean mechanism story rather than a clean algorithmic win. Process-aware shaping can indeed move the learned policy toward ambiguity-focused verification. But the learned controller overfits the short-horizon budget regime: it learns to check, yet fails to translate that checking into a reliable longer-chain repair-and-commit strategy.

Cue-quality robustness supports that interpretation. When we evaluate the held-out policies under cleaner cues (fault-step hit rate 0.95, false-positive rate 0.05), the hint-triggered heuristic rises to 0.939 ± 0.005 success and monitorability-aware RL rises modestly to 0.137 ± 0.038, with failure localization 0.629 ± 0.093 (Appendix Table 3). Under noisier cues (0.60/0.35), the heuristic drops sharply to 0.512 ± 0.017 and monitorability-aware RL drops to 0.083 ± 0.037, with localization falling to 0.441 ± 0.061. The learned policy therefore does use the cue, but it converts better cues into diagnostics more reliably than into successful longer-horizon repair.

The held-out ablations support that interpretation. Removing the budget penalty raises held-out success from 0.112 ± 0.035 to 0.291 ± 0.090, though cost-adjusted accuracy remains negative. Removing the cue/gap monitorability extras raises held-out success only modestly to 0.141 ± 0.053, but also cuts failed-episode localization from 0.556 ± 0.083 to 0.305 ± 0.138. By contrast, removing `VERIFY` collapses held-out success to zero, while removing `REVISE` yields only 0.139 ± 0.019. The full proposed shaping is therefore best viewed as an in-

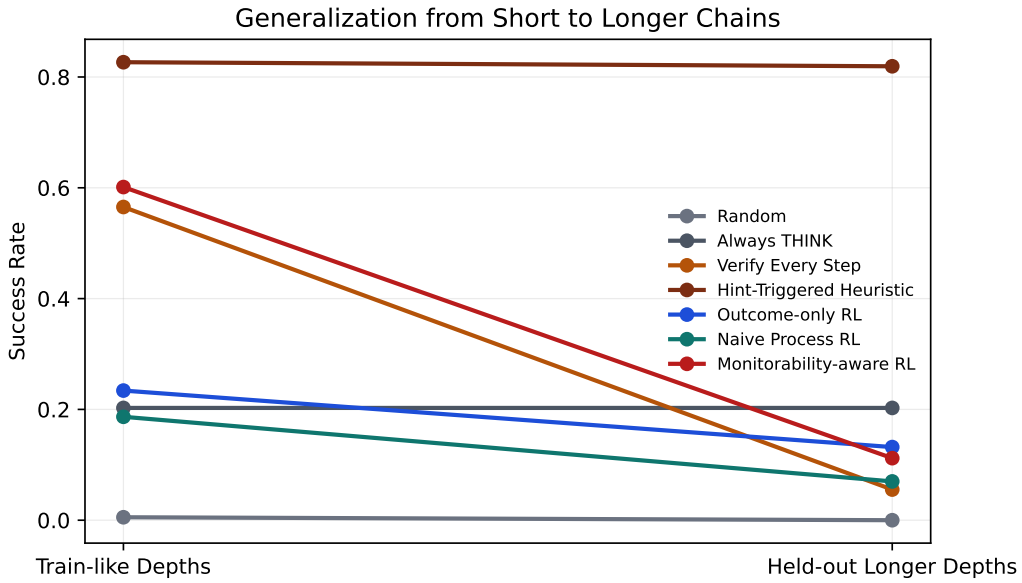


Figure 2: Generalization from short training chains to longer held-out chains. The monitorability-aware policy learns a stronger in-distribution checking strategy, but this behavior does not transfer robustly to longer reasoning chains.

distribution inducer of checking behavior that trades away held-out efficiency for much stronger diagnostics.

6.4 Cost and budget sensitivity remain sharp

The held-out tradeoff plot in Figure 3 shows that verification remains budget-sensitive even after adding the ambiguity cue. Increasing verification cost quickly degrades the learned ambiguity-checking policy. Likewise, more budget helps the hint-triggered heuristic substantially, but the learned monitorability-aware policy does not fully capitalize on that slack. This is consistent with the broader interpretation of the paper: the environment makes the intended policy available, yet small-scale RL still struggles to learn a version that is both selective and robust.

7 Discussion

The point of this environment is not its raw difficulty. The useful property is that it isolates a concrete decision: when the current local step looks ambiguous, should the controller spend scarce budget to verify now, move forward, or backtrack? Because the environment has exact ground truth, we can distinguish three things cleanly: whether the controller solved the task, whether it checked the right local steps, and whether failures stayed locally diagnosable.

In that sense the paper succeeds as a mechanism study. The ambiguity cue is enough to support a strong selective-verification heuristic. A monitorability-aware reward can push a learned policy toward that kind of checking. But the learned controller still fails to recover the heuristic robustly, especially on longer chains. This makes the project a better fit for Concept & Feasibility or Negative Results than for a strong main-track method claim.

8 Limitations

This work remains intentionally tiny. The controller is a small MLP over a symbolic state, not a language model; the environment contains at most one hidden fault; the ambiguity cue

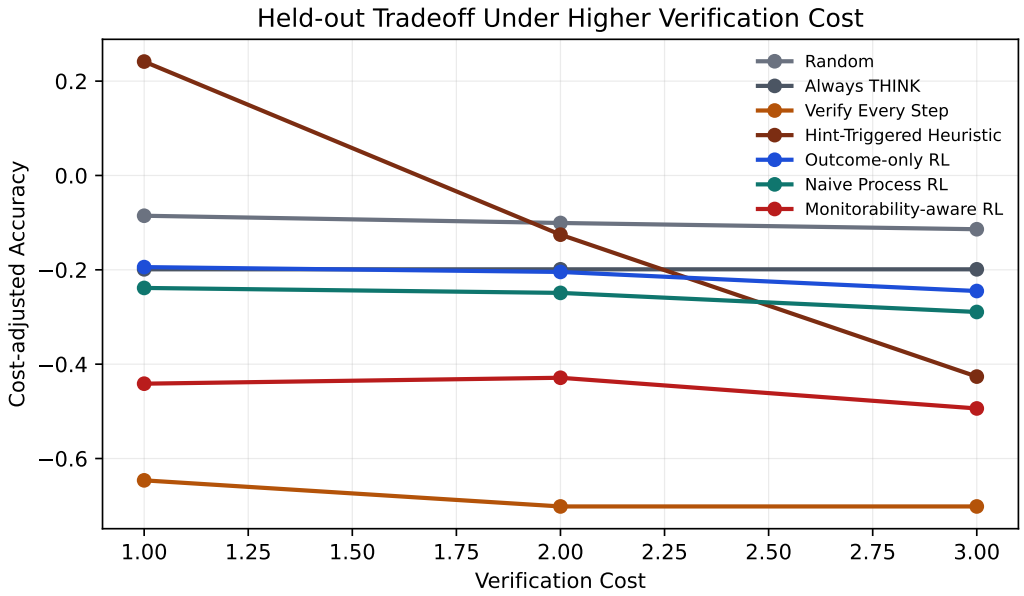


Figure 3: Held-out cost-adjusted accuracy versus verification cost. The learned process-aware policy is still brittle: stronger ambiguity checking does not automatically translate into robust longer-horizon control.

is hand-designed; and the local arithmetic-chain abstraction does not capture the semantic ambiguity or strategic concealment that matter in real chain-of-thought settings. The hint-triggered heuristic also shows that part of the difficulty here is simply learning a short program from small-budget RL, not discovering a fundamentally unavailable strategy.

The empirical limitation is more important: the learned monitorability-aware policy does look promising in-distribution, but it still fails under a simple length shift. The strongest honest conclusion is not that the proposed reward solves budgeted process supervision, but that it reveals a sharp overfitting mode in tiny process-aware RL. A related societal risk is over-interpretation: toy monitorability gains in exact symbolic settings could be mistaken for evidence that natural-language chain-of-thought is faithful or robustly monitorable in realistic systems. This paper should not be read that way.

9 Conclusion

We introduced a tiny symbolic environment for studying budgeted process supervision with exact local checking, backtracking, and noisy ambiguity cues. In this proxy, ambiguity-aware process rewards can induce in-distribution checking behavior, but the resulting policies remain brittle under longer held-out chains. The main contribution is therefore diagnostic: exact small-scale environments can reveal how process rewards shape verification behavior, and can show clearly when those gains fail to become robust reasoning improvements.

References

Anthropic. Reasoning models don't always say what they think. `https://www.anthropic.com/research/reasoning-models-dont-say-think`, April 2025. Accessed April 10, 2026.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL `https://arxiv.org/abs/2501.12948`.

OpenAI. Learning to reason with LLMs. https://openai.com/index/learning-to-reason-with-llms/, September 2024. Accessed April 10, 2026.

OpenAI. Evaluating chain-of-thought monitorability. https://openai.com/index/evaluating-chain-of-thought-monitorability/, December 2025. Accessed April 10, 2026.

Tian Xie, Zitian Gao, Qingnan Ren, Haoming Luo, Yuqian Hong, Bryan Dai, Joey Zhou, Kai Qiu, Zhirong Wu, and Chong Luo. Logic-RL: Unleashing LLM reasoning with rule-based reinforcement learning. *arXiv preprint arXiv:2502.14768*, 2025. URL https://arxiv.org/abs/2502.14768.

A Additional Experimental Details

Training uses a tiny masked Double DQN with 64 hidden units per layer and Adam with learning rate 10^{-3}. The observation dimension is 10, the action space has size 4, and the default controller has 5,124 trainable parameters. The full run trains 35 learned models (7 learned variants times 5 seeds), each for 5,000 episodes, for a total of 175,000 training episodes. Evaluation uses 12 settings, 11 methods, 5 seeds, and 300 episodes per seed, for a total of 198,000 evaluation episodes. All tasks are procedurally generated online.

Table 2: Default environment and learner hyperparameters used for the main study.

Quantity	Value
Training depths	3–6
Held-out depths	7–9
Fault probability	0.8
Hint hit / false-positive rates	0.85 / 0.15
Budget	depth +6 actions
Action costs (`THINK`, `VERIFY`, `REVISE`)	$(1, 1, 1)$
Network	10–64–64–4 ReLU MLP
Trainable parameters	5,124
Optimizer / learning rate	Adam / 10^{-3}
Discount factor	0.99
Replay size / batch size	25,000 / 64
Episodes per run / seeds	5,000 / 5
$(\lambda_{\text{safe}}, \lambda_{\text{fault}}, \lambda_{\text{hint}}, \lambda_{\text{waste}}, \lambda_{\text{gap}})$	$(0.02, 0.25, 0.18, 0.06, 0.30)$

B Behavior-Level Diagnostics

Figure 4 reports cue-conditioned verification rates. This is the most direct diagnostic of the paper's mechanism claim: monitorability-aware RL should verify *selectively*, not merely verify more often. The figure shows that the learned policy does become cue-sensitive in-distribution and remains partially cue-sensitive under the held-out length shift, whereas outcome-only RL is much less selective.

C Cue-Quality Robustness

Table 3 evaluates held-out longer chains under cleaner and noisier ambiguity cues *without retraining*. This isolates whether the learned policy actually relies on the local cue. Cleaner cues help both the heuristic and the monitorability-aware policy, while noisier cues hurt both, which supports the interpretation that the process-aware policy does learn cue-dependent behavior. But the learned policy turns improved cue quality into better localization more reliably than into better success.

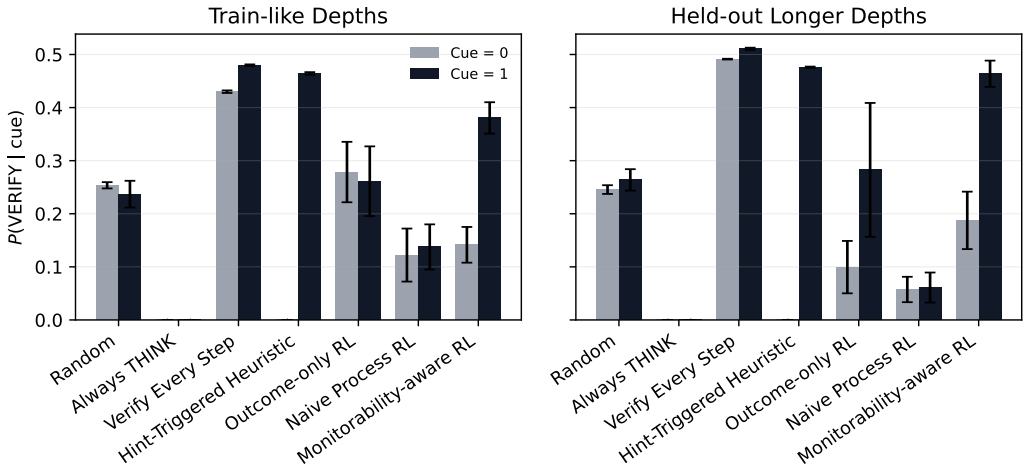


Figure 4: Cue-conditioned verification rates. The proposed reward induces more selective checking than outcome-only RL, but still falls short of the hand-coded hint-triggered heuristic, especially under the held-out shift.

Table 3: Held-out longer-chain performance under cleaner and noisier ambiguity cues, without retraining. Entries are mean ± standard error over five seeds.

Method	CleanSucc	NoisySucc	CleanLoc	NoisyLoc
Hint-Triggered Heuristic	0.939 ± 0.005	0.512 ± 0.017	0.433 ± 0.030	0.368 ± 0.008
Outcome-only RL	0.143 ± 0.062	0.120 ± 0.049	0.111 ± 0.052	0.107 ± 0.041
Naive Process RL	0.076 ± 0.032	0.061 ± 0.014	0.149 ± 0.052	0.184 ± 0.063
Monitorability-aware RL	0.137 ± 0.038	0.083 ± 0.037	0.629 ± 0.093	0.441 ± 0.061

D External Assets

The paper uses the official NeurIPS 2026 style bundle from `https://neurips.cc`, the `lineno` package under the LaTeX Project Public License, and the Latin Modern font package under the GUST Font License / LPPL terms. The experiments use only the Python dependencies listed in `requirements.txt`; no external datasets, pretrained models, or proprietary APIs are required. For anonymous submission to a workshop or conference, code and artifacts should be shared through an anonymized supplement or anonymous repository link rather than the named development repository used during project preparation.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction explicitly present the work as a tiny symbolic proxy and state the main result as an in-distribution monitorability gain that does not generalize to longer chains.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and

limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.

- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section "Limitations" explicitly discusses the narrow symbolic environment, hand-designed ambiguity cue, tiny masked Double DQN controller, and the weak out-of-distribution results.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper is empirical and does not claim a formal theorem or proof.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.

- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Sections "Problem Setup," "Method," "Experimental Setup," and the appendix specify the environment, reward functions, learner, seeds, and evaluation protocol, and the repository includes runnable scripts and raw CSV outputs.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
     (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The project uses no external dataset, and the repository includes the full source, configs, scripts, figures, tables, and reproduction commands described in the appendix and README.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Section "Experimental Setup" gives the train/test depth split, budgets, reward settings, seed count, and hardware, while the appendix records the masked Double DQN and reward hyperparameters.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Tables and figures report mean ± standard error over five random seeds, and the experimental setup section states the seed count and evaluation protocol.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Section "Experimental Setup" and Appendix Table 2 report the Apple M1 laptop, CPU compatibility, model size, training and evaluation episode counts, and repository footprint.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The project uses only synthetic arithmetic tasks, does not involve human subjects, personal data, external APIs, or unsafe model release, and is presented with explicit limitations.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section "Limitations" notes both the value of cheap exact testbeds and the negative risk of over-interpreting toy monitorability results as evidence about real model faithfulness.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The release contains only small symbolic environment code, figures, and paper assets; it does not release a pretrained model, scraped dataset, or high-risk artifact.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The appendix names the external assets used directly in the submission package, including the official NeurIPS style bundle, the `lineno` package under LPPL, and Latin Modern under the GUST Font License / LPPL terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new assets are the environment, training/evaluation scripts, result CSVs, figures, and paper source; they are documented in the paper, appendix, and repository README.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The project contains no human-subject data collection or experimentation.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: No large language model is part of the reported environment, learner, reward design, or evaluation pipeline; the core method is a symbolic environment with a tiny masked Double DQN.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.